

Bài thực hành buổi 4

Học Phần : Xử Lý Ảnh

1. Mục tiêu:

- Thực hành các phương pháp lọc đường biên: Bộ lọc Gradient, Bộ lọc Robert, Bộ lọc Sobel, lọc Sobel và lọc Laplace
- Thực hành phương pháp phân ngưỡng tự động bằng giải thuật tuần tự và giải thuật Otsu

2. Thực hành:

Sinh viên tiến hành thiết kế giao diện như hình

The screenshot shows a software window titled "MainWindow" with standard Windows window controls (minimize, maximize, close). The interface is organized into two main panels. The top panel, labeled "Xử lý 1 ảnh", contains several input fields and buttons for image processing. It includes a "Tên file ảnh" field with a "Chọn" button, an "Ảnh âm bản" section with "Xử lý ảnh xám" and "Xử lý ảnh màu" buttons, a "Mức sáng" field with a range "> 0 tăng; < 0 giảm" and corresponding buttons, a "Độ tương phản" field with a range "> 1 tăng; 0 < c < 1 giảm" and corresponding buttons, a "Chọn ngưỡng" field with a range "> n trắng; < n đen" and buttons for "Xử lý phân ngưỡng", "Giải thuật tuần tự", and "Giải thuật Otsu". There are also buttons for "Vẽ histogram ảnh xám", "Vẽ histogram ảnh màu", "Biến đổi tuyến tính histogram", and "Cân bằng histogram". At the bottom of this panel, there are buttons for "Kích thước mặt nạ" and "Tỷ Lệ nhiễu (%)", along with buttons for "Lọc trung bình", "Lọc trung vị", "Gauss + Laplace", "Thêm nhiễu muối tiêu", "Thêm nhiễu ảnh màu", and "G + L Ảnh Màu". A red box highlights a group of buttons for edge detection and gradient filters: "Lọc Gradient", "Lọc Robert", "Lọc Sobel", "Lọc Prewitt", "Lọc Laplace", "Gradient ảnh màu", "Lọc Robert ảnh màu", "Lọc Sobel ảnh màu", "Lọc Prewitt ảnh màu", and "Lọc Laplace ảnh màu". The bottom panel, labeled "Xử lý 2 ảnh", contains two "Chọn ảnh" buttons, two "Opacity ảnh" fields, and buttons for "Cộng 2 ảnh" and "Trừ 2 ảnh".

2.1) Lọc đường biên bằng phương pháp Gradient

Tại sự kiện clicked của nút “Lọc Gradient”, sinh viên tiến hành nhập vào đoạn code sau:

```
void MainWindow::on_btn_gradient_clicked()
{
    QImage image_in(ui->LineFileName->text());
    QImage image_out_Gx(image_in.width()-1,image_in.height(),QImage::Format_RGB32);
    QImage image_out_Gy(image_in.width(),image_in.height()-1,QImage::Format_RGB32);

    for (int i = 0; i < image_in.width() - 1; i++)
    {
        for(int j = 0; j < image_in.height(); j++)
        {
            QRgb color1 = image_in.pixel(i,j);
            int gray1 = qGray(color1);

            QRgb color_in = qRgb(gray1,gray1,gray1);
            image_in.setPixel(i,j,color_in);

            QRgb color2 = image_in.pixel(i+1,j);
            int gray2 = qGray(color2);
            int newgray = abs (gray2 - gray1);
            image_out_Gx.setPixel(i,j,qRgb(newgray,newgray,newgray));
        }
    }
    for (int i = 0; i < image_in.width(); i++)
    {
        for(int j = 0; j < image_in.height()-1; j++)
        {
            QRgb color1 = image_in.pixel(i,j);
            int gray1 = qGray(color1);

            QRgb color_in = qRgb(gray1,gray1,gray1);
            image_in.setPixel(i,j,color_in);

            QRgb color2 = image_in.pixel(i,j+1);
            int gray2 = qGray(color2);
            int newgray = abs (gray2 - gray1);
            image_out_Gy.setPixel(i,j,qRgb(newgray,newgray,newgray));
        }
    }

    QLabel* label_in = new QLabel();
    label_in->setPixmap(QPixmap::fromImage(image_in.scaled(1024,768)));
    label_in->show();

    QLabel *label_Gx = new QLabel();
    label_Gx->setPixmap(QPixmap::fromImage(image_out_Gx.scaled(1024,768)));
    label_Gx->show();

    QLabel *label_Gy = new QLabel();
    label_Gy->setPixmap(QPixmap::fromImage(image_out_Gy.scaled(1024,768)));
    label_Gy->show();
}
```

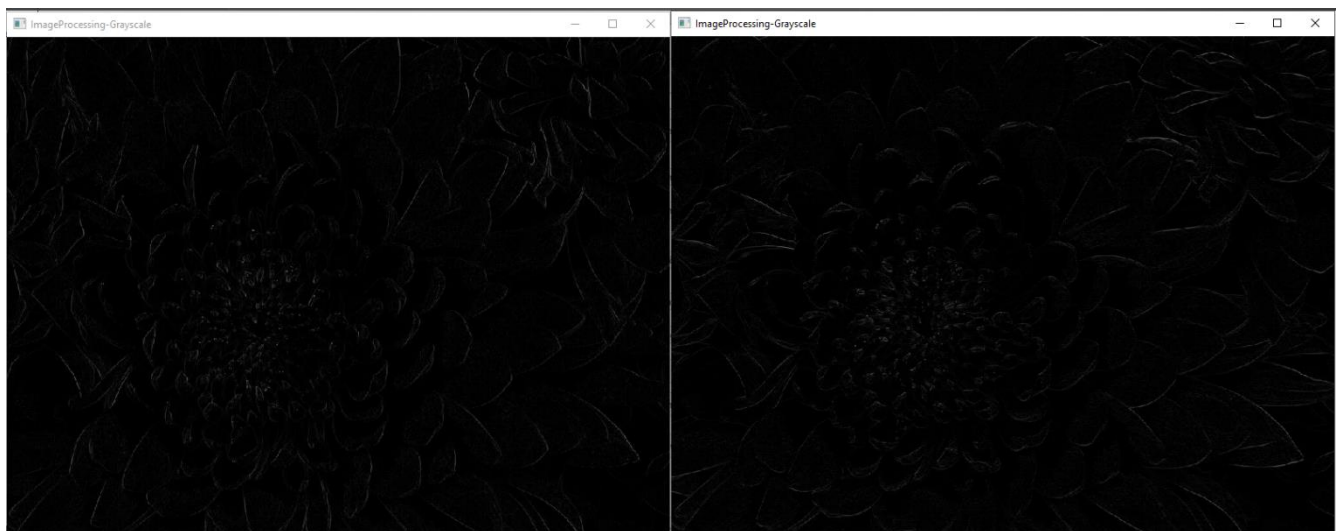
Đoạn code trên sẽ tiến hành tính đạo hàm Gx và Gy của ảnh đầu vào và xuất ra màn hình thành 2 ảnh phân biệt cho người dùng.

Để hiểu rõ nguyên lý sinh viên cần xem lại phần lý thuyết về nguyên lý lọc Gradient của chương 5 – Phát hiện đường biên.

Kết quả thu được :



Ảnh gốc



Gradient Gx

Gradient Gy

2.2) Lọc đường biên bằng bộ lọc Robert

Nhắc lại kiến thức : Bộ lọc Robert là một bộ lọc đặc biệt có kích thước 2x2 với tác dụng làm nổi bật những đường biên theo hướng chéo

$$\begin{array}{cc} \boxed{0} & 1 \\ -1 & 0 \end{array} \quad \begin{array}{cc} \boxed{1} & 0 \\ 0 & -1 \end{array}$$

$\swarrow$ $\nwarrow$

Gx Gy

Tại sự kiện clicked của nút “Lọc Robert” , ta thêm vào đoạn code sau :

```

void MainWindow::on_btn_robert_clicked()
{
    QImage image_in(ui->LineFileName->text());
    QImage image_out_Gx(image_in.width()-1,image_in.height()-1,QImage::Format_RGB32);
    QImage image_out_Gy(image_in.width()-1,image_in.height()-1,QImage::Format_RGB32);

    for (int i = 0; i <image_in.width()-1 ; i++)
    {
        for (int j = 0; j<image_in.height()-1 ; j ++)
        {
            QRgb color1 = image_in.pixel(i,j);
            int gray1 = qGray(color1);
            QRgb color2 = image_in.pixel(i+1,j);
            int gray2 = qGray(color2);
            QRgb color3 = image_in.pixel(i,j+1);
            int gray3 = qGray(color3);
            QRgb color4 = image_in.pixel(i+1,j+1);
            int gray4 = qGray(color4);

            int newgray_gx = abs(gray1 - gray4);

            int newgray_gy = abs(gray2 - gray3);

            image_out_Gx.setPixel(i,j,qRgb(newgray_gx,newgray_gx,newgray_gx));
            image_out_Gy.setPixel(i,j,qRgb(newgray_gy,newgray_gy,newgray_gy));
        }
    }

    QLabel *label_in = new QLabel();
    label_in->setPixmap(QPixmap::fromImage(image_in.scaled(1024,768)));
    label_in->show();

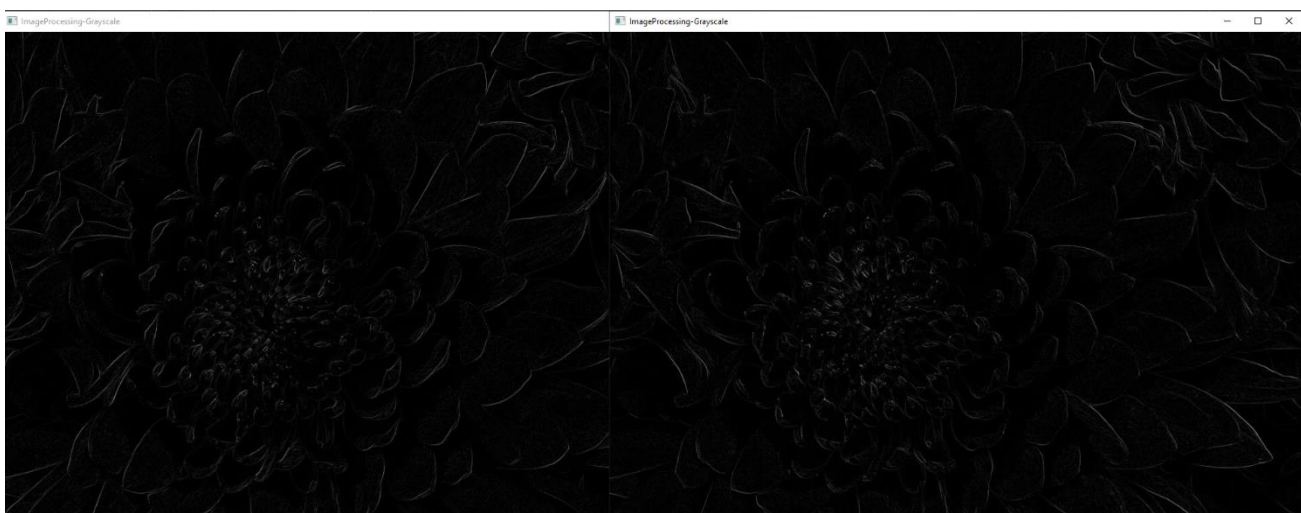
    QLabel *label_Gx = new QLabel();
    label_Gx->setPixmap(QPixmap::fromImage(image_out_Gx.scaled(1024,768)));
    label_Gx->show();

    QLabel *label_Gy = new QLabel();
    label_Gy->setPixmap(QPixmap::fromImage(image_out_Gy.scaled(1024,768)));
    label_Gy->show();
}

```

Đoạn chương trình trên sẽ tìm ra các đường biên theo 2 hướng chéo và lưu vào các ảnh Gx và Gy. Sinh viên hãy liên hệ với bộ lọc Robert để hiểu hơn đoạn chương trình trên.

Kết quả thu được :



Robert Gx

Robert Gy

2.3) Lọc đường biên bằng bộ lọc Sobel

Nhắc lại kiến thức : bộ lọc Sobel là sự kết hợp của việc làm trơn ảnh và lấy đạo hàm của ảnh

Sobel:

-1	-2	-1	-1	0	1
0	0	0	-2	0	2
1	2	1	-1	0	1

Ta thêm đoạn code sau vào sự kiện clicked của nút “ Lọc Sobel ” :

```
void MainWindow::on_btn_sobel_clicked()
{
    QImage image_in(ui->LineFileName->text());
    QImage image_out(image_in.width(),image_in.height(),QImage::Format_RGB32);
    QImage image_out2(image_in.width(),image_in.height(),QImage::Format_RGB32);

    int maskSize = 3;
    int margin = maskSize / 2;
    //int cells = maskSize * maskSize;

    int mask[maskSize][maskSize];
    mask[0][0] = -1; mask[0][1] = 0; mask[0][2] = 1;
    mask[1][0] = -2; mask[1][1] = 0; mask[1][2] = 2;
    mask[2][0] = -1; mask[2][1] = 0; mask[2][2] = 1;
    int mask2[maskSize][maskSize];
    mask2[0][0] = -1; mask2[0][1] = -2; mask2[0][2] = -1;
    mask2[1][0] = 0; mask2[1][1] = 0; mask2[1][2] = 0;
    mask2[2][0] = 1; mask2[2][1] = 2; mask2[2][2] = 1;
    QRgb color;
    int sum1, sum2;
    for (int y = margin; y < image_in.height() - margin; y++)
        for (int x = margin; x < image_in.width() - margin; x++)
        {
            sum1=sum2 = 0;
            for (int j= -margin; j <= margin; j++)
                for (int i = -margin; i <= margin; i++)
                {
                    color = image_in.pixel(x + i, y + j);
                    int gray = qGray(color);
                    sum1 += gray * mask[j+margin][i+margin];
                    sum2 += gray * mask2[j+margin][i+margin];
                }
            image_out.setPixel(x, y, qRgb(abs(sum1), abs(sum1), abs(sum1)));
            image_out2.setPixel(x, y, qRgb(abs(sum2), abs(sum2), abs(sum2)));
        }

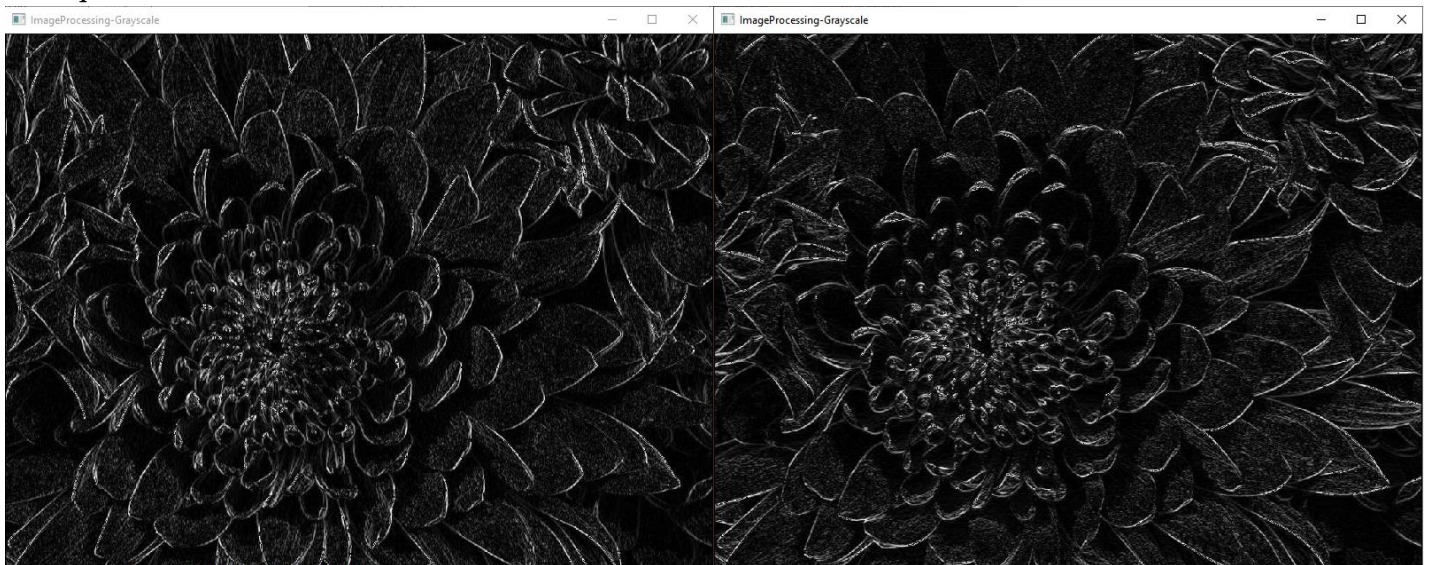
    QLabel *label_in = new QLabel();
    label_in->setPixmap(QPixmap::fromImage(image_in.scaled(1024,768)));
    label_in->show();

    QLabel *label_Gx = new QLabel();
    label_Gx->setPixmap(QPixmap::fromImage(image_out.scaled(1024,768)));
    label_Gx->show();

    QLabel *label_Gy = new QLabel();
    label_Gy->setPixmap(QPixmap::fromImage(image_out2.scaled(1024,768)));
    label_Gy->show();
}
```

Đoạn chương trình trên sẽ tiến hành tính tích chập của ảnh với 2 bộ lọc Sobel để thu được kết quả Gx và Gy. Sinh viên so sánh kết quả với các bộ lọc trước đó và đưa ra kết luận.

Kết quả thu được :



Sobel Gx

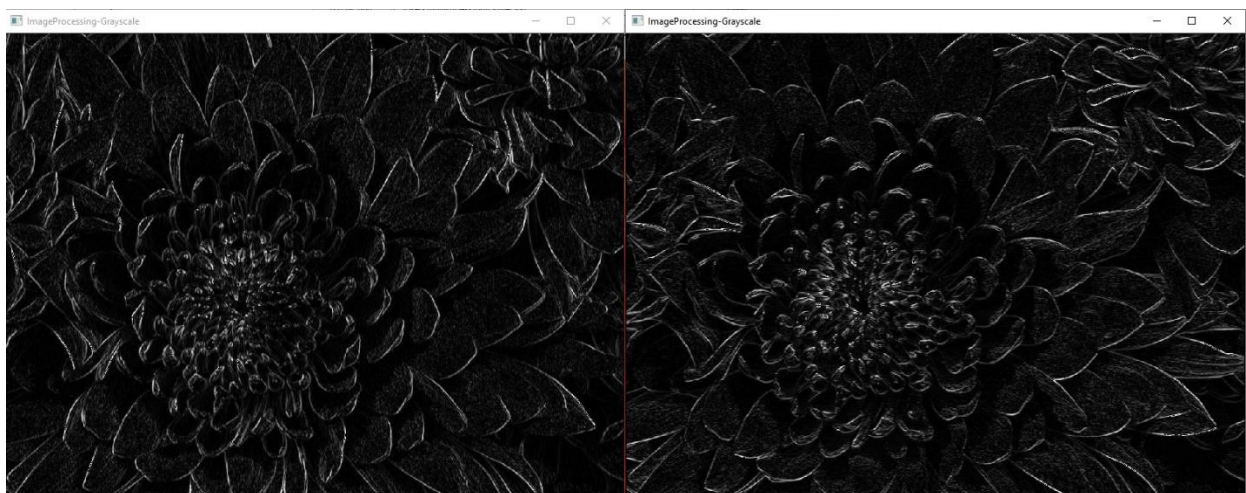
Sobel Gy

3) Bài tập

Bài 1 : Thiết kế chức năng cho bộ lọc Prewitt cho ảnh xám

-1	-1	-1
0	0	0
1	1	1

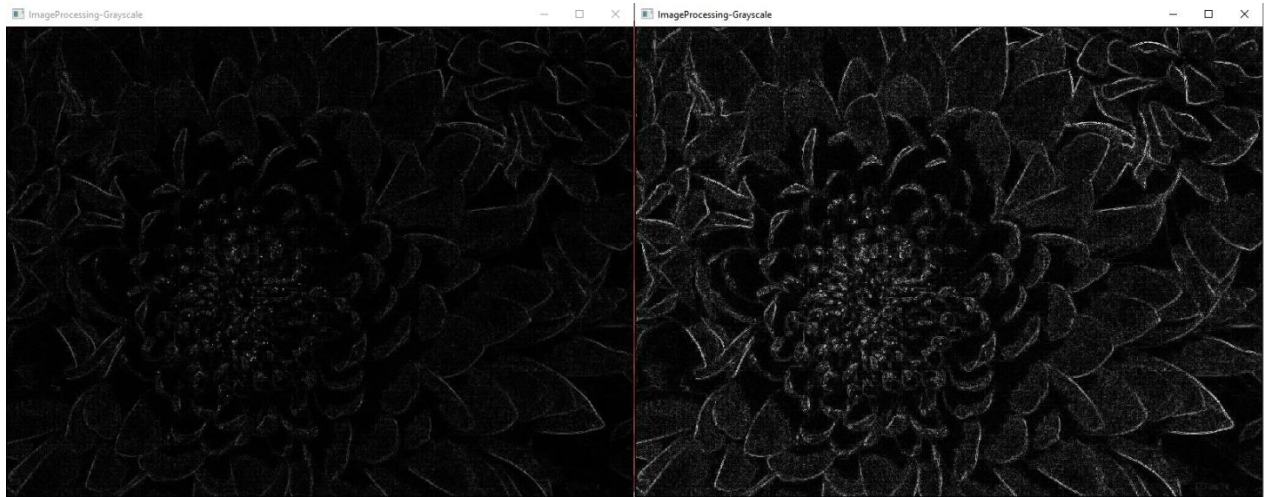
-1	0	1
-1	0	1
-1	0	1



Bài 2 : Thiết kế chức năng cho bộ lọc Laplace cho ảnh xám

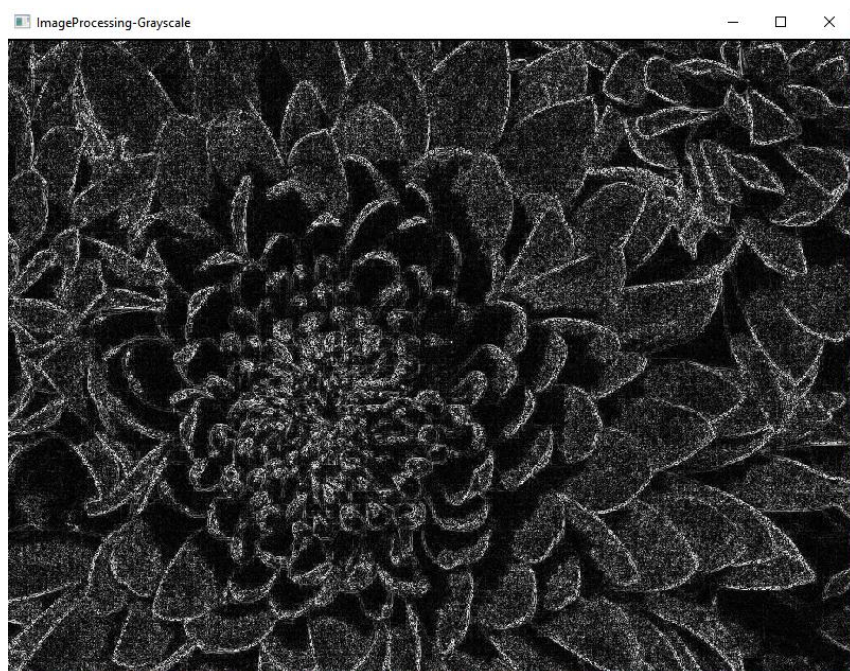
0	1	0
1	-4	1
0	1	0

1	1	1
1	-8	1
1	1	1

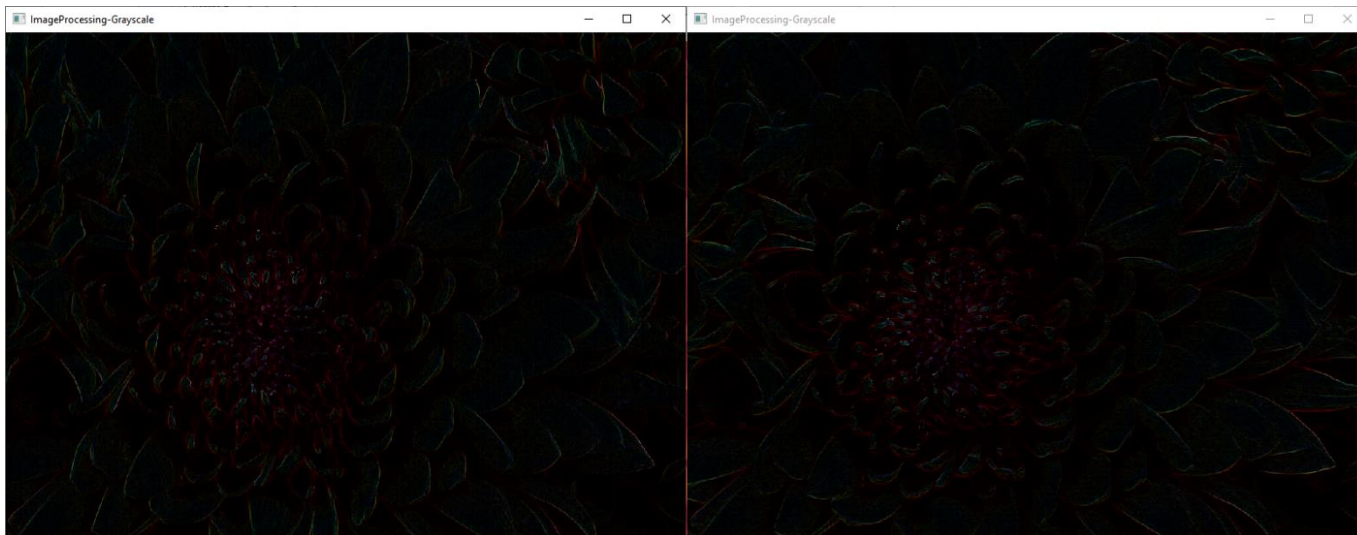


Bài 3 : Thiết kế tính năng cho Bộ lọc Gauss kết hợp Laplace cho ảnh xám

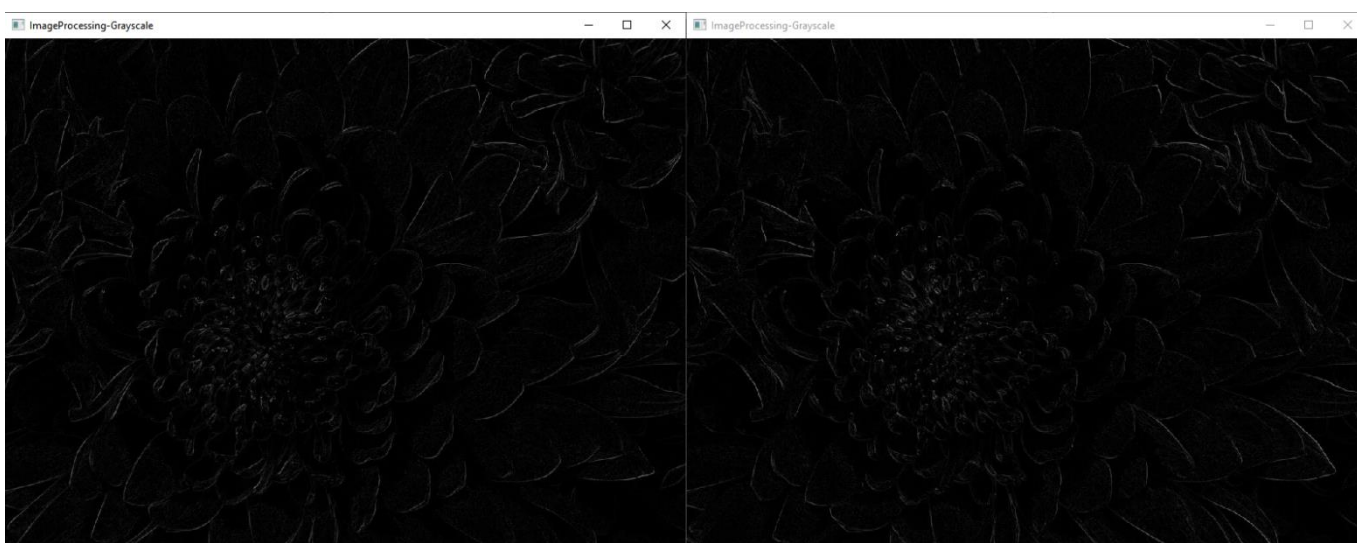
0	0	-1	0	0
0	-1	-2	-1	0
-1	-2	16	-2	-1
0	-1	-2	-1	0
0	0	-1	0	0



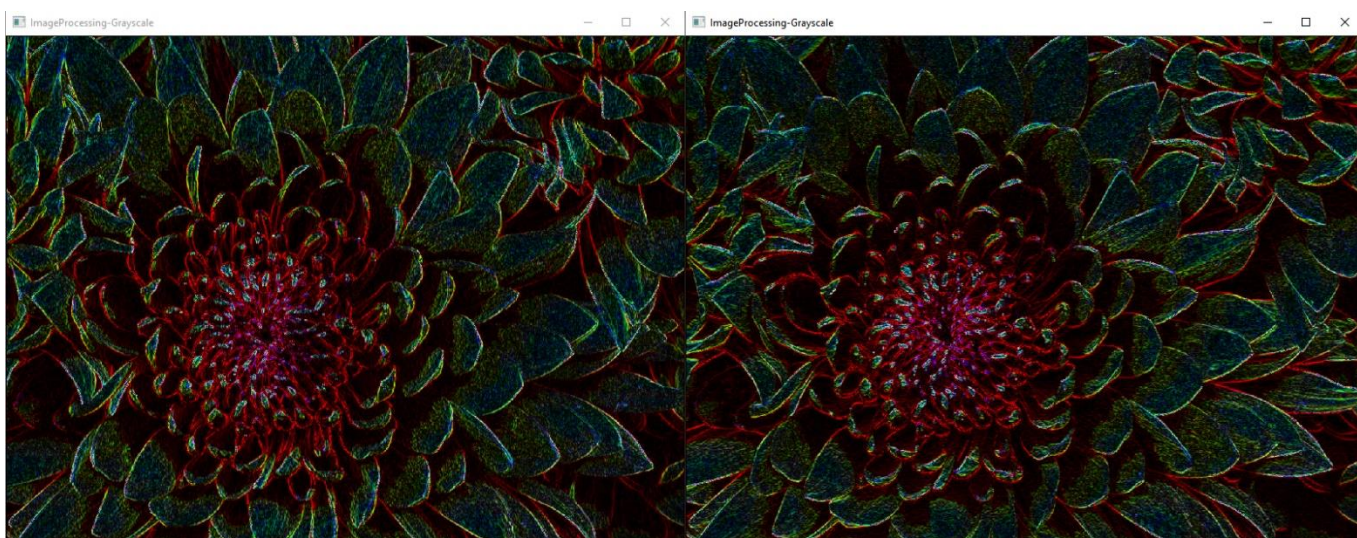
Bài 4 : Thiết kế tính năng Lọc Gradient cho ảnh màu



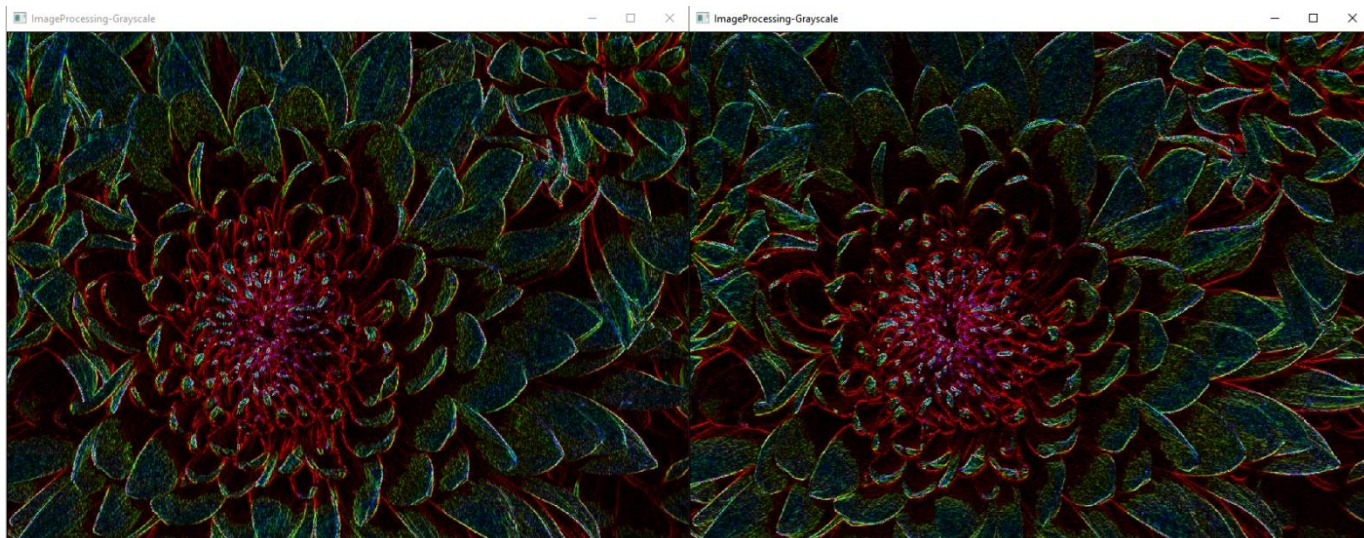
Bài 5 : Thiết kế tính năng Lọc Robert cho ảnh màu



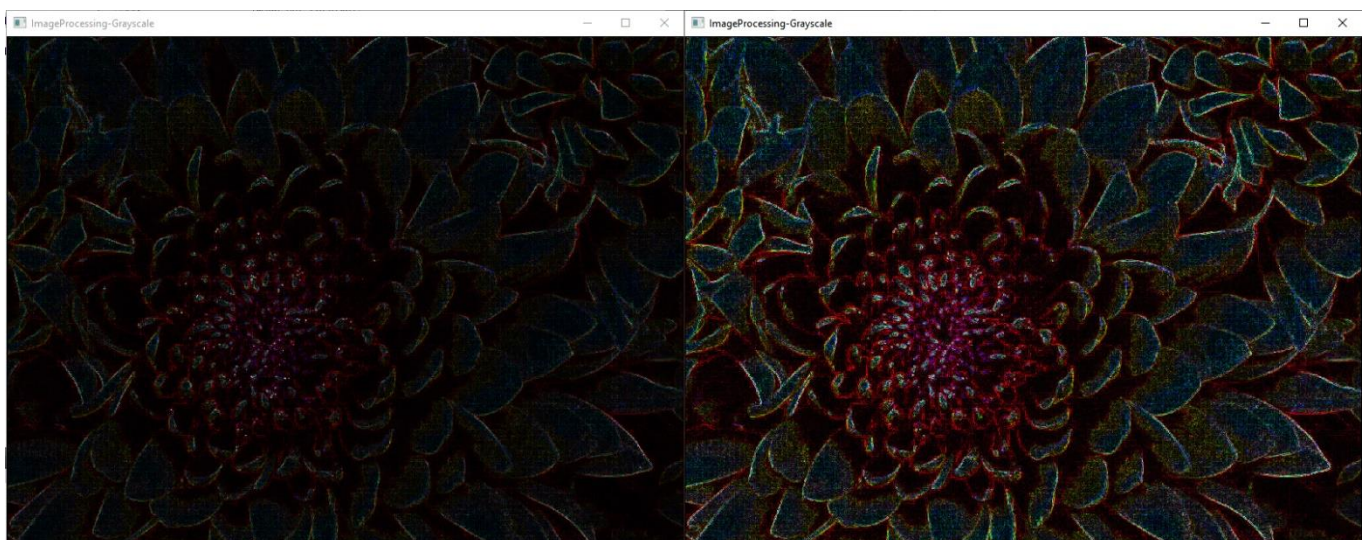
Bài 6 : Thiết kế tính năng Lọc Sobel cho ảnh màu



Bài 7 : Thiết kế tính năng Lọc Prewitt cho ảnh màu



Bài 8 : Thiết kế tính năng Lọc Laplace cho ảnh màu



Bài 9 : Thiết kế tính năng Lọc Gauss kết hợp Laplace cho ảnh màu

